

KAVALAR

AI Governance & Behavioral Anomaly Detection
for LLM Tool Executions

Technical Solution & Architecture Write-up

Problem Domain:	Indirect Prompt Injection & LLM Agent Security
Technical Approach:	Isolation Forest Machine Learning + Deterministic Rules Override
Framework Stack:	FastAPI, LangGraph, Groq, Next.js, SQLite/PostgreSQL
Date of Publication:	July 04, 2026
Target Review Group:	AI Security Governance & Verification Team

CONFIDENTIAL - Distributed solely for independent team review.

1. Executive Summary & Problem Statement

Kavalar: State-of-the-Art Anomaly Detection & Prompt Injection Defense

Large Language Model (LLM) agents are increasingly integrated into enterprise workflows, utilizing framework graphs like LangGraph to execute critical functions via bound tools (e.g., retrieving files, looking up client profiles, sending emails, or running math equations). However, this tool-based agency exposes LLMs to a critical vulnerability: Indirect Prompt Injection (IPI).

In an Indirect Prompt Injection attack, an attacker does not query the LLM directly. Instead, they poison an external data source, such as a database row, customer profile, or public document. When the agent reads this poisoned resource using a standard tool, the malicious payload is injected into the LLM's context window. The payload instructs the LLM to override its system instructions and misuse other tools - for example, exfiltrating sensitive client details via an email tool or performing looping database scraping.

Why Traditional Security Scanners Fail:

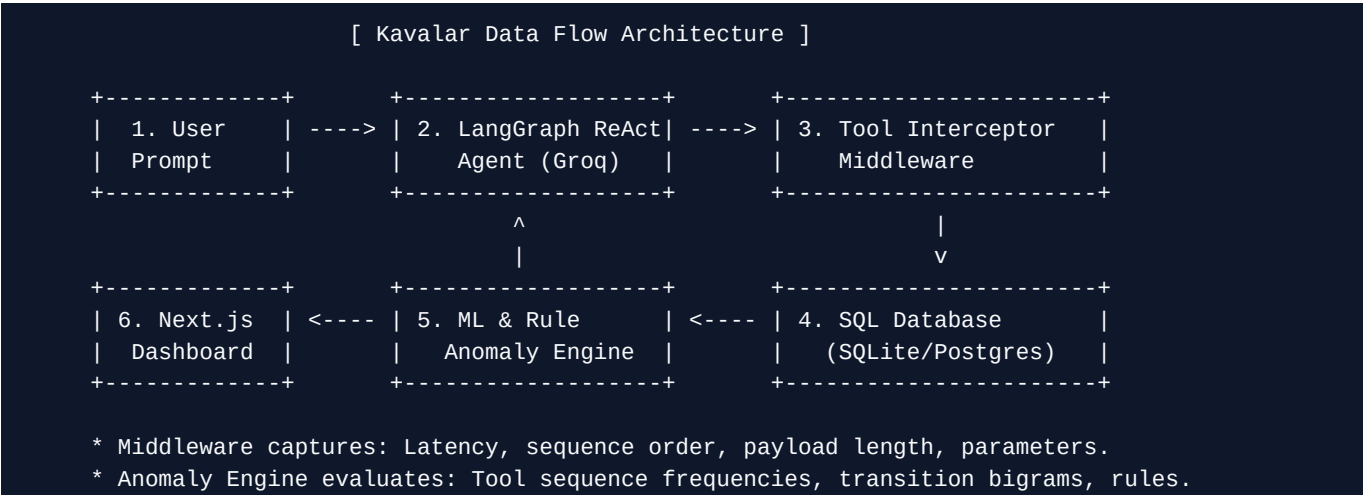
1. Semantics are bypassable: Prompt injections use natural language, obfuscation, or translation to slip past regex filters.
2. Input size limitations: High-volume scraping loops or prompt overrides are only evident during step-by-step executions.
3. System context loss: Input filters inspect isolated text blocks, missing the holistic behavior of tool sequence transitions.

The Kavalar Defense Concept

Kavalar introduces an out-of-band behavioral security guardrail. Instead of parsing textual inputs, Kavalar treats the agent's tool invocations as an execution vocabulary. By profiling normal agent runs, Kavalar establishes a behavior baseline. When an agent session runs, Kavalar captures execution duration, sequence order, parameter sizes, and tool transitions. Any deviation from the baseline is flagged in real time using an Isolation Forest machine learning model combined with deterministic security rules.

2. Technical Solution & System Architecture

Kavalar's architecture consists of a LangGraph agent loop monitored by an auditing middleware interceptor sidecar. Execution logs are stored in a SQL database (SQLite/PostgreSQL), processed by a feature engineering engine, and classified by the machine learning/rules engine.



Core System Modules:

1. LangGraph State ReAct Agent:

Runs reasoning loops utilizing llama-3.3-70b-versatile. It binds five tools: search_documents, read_document, calculator, send_email, lookup_customer.

2. Interceptor Middleware Decorator:

Wraps tools in a sidecar auditor. It tracks execution times, sequence counters, parameters, and writes records directly to the SQL database.

3. Isolation Forest ML Engine:

Calculates numerical session vectors mapping frequency counts, sequence lengths, average execution times, and parameters. The decision score is sigmoid-scaled into a probability value between 0.0 and 1.0.

4. Behavior Explainer (Rule Layer):

A concurrent deterministic rule processor that verifies vocabulary sanity, sequence transitions, and matches injection regex vectors to act as an override.

3. Mathematical Modeling & ML Details

To perform machine learning predictions on discrete tool sequences, Kavalar converts each session into a multi-dimensional numerical feature vector. The baseline training profiles the unique tools used in normal operations to establish a vocabulary.

Feature Vector Construction

For any agent session S , the behavior vector X is formulated as:

$$X = [f, c, t_avg, l_param, b]$$

Where:

- f (Tool Frequency Vector): M -dimensional counts of baseline vocabulary tools in the session.
- c (Execution Count): Total number of tool invocations.
- t_avg (Average Execution Time): Mean latency of tool executions in the session.
- l_param (Parameter Length): Character length of stringified JSON tool arguments.
- b (Transition Bigrams): Count of adjacent tool-to-tool transitions in the session sequence.

Isolation Forest Scoring and Sigmoid Scaling

The scikit-learn Isolation Forest produces a raw anomaly decision score ' d ' centred around 0.0, where negative scores indicate outliers (anomalies) and positive scores indicate normal profiles. To make this result intuitive for security engineers, Kavalar scales the raw score using a Sigmoid function:

$$\text{Probability(Injection)} = 1 / (1 + \exp(d * 10.0))$$

This scales the anomaly rating into a probability P in $[0.0, 1.0]$:

- * If $d \leq 0$ (Anomalous), the scaled probability $P \geq 0.5$
- * If $d > 0$ (Normal), the scaled probability $P < 0.5$

Deterministic Rules & Overrides

Since machine learning models can exhibit false negatives, Kavalar runs an Explainability Rules engine side-by-side. If a high-confidence rule fires, the engine immediately overrides the Isolation Forest label, sets the session status to 'Injected', and boosts the threat probability score above 0.5. Rules cover:

- Unexpected Tool: Session contains tools not in the vocabulary V (e.g. `send_email`).
- Sequence Transition: A tool transition bigram is observed that was never seen in training.
- Execution Frequency: The tool call count exceeds the training maximum by more than 2 standard deviations.
- Parameter Payload Size: The string size of arguments is abnormally large, indicating embedded prompts.
- Signature Patterns: Argument strings match SQL injection, path traversal, or script tag regexes.

4. Threat Simulations & Real-World Outcomes

To test and verify the detector, Kavalar integrates a simulation sandbox featuring four pre-configured scenarios representing typical agent tasks and threat vectors.

ID	Scenario Name	Description & Tool Sequence	Expected Status
0	Normal Run	Lookup customer & run travel expense math. Sequence: lookup_customer -> calculator	Normal
			(P < 0.10)
1	Exfiltration	Attacker poisons file with instructions to email database keys. Sequence: search_documents -> send_email	Injected
			(P >= 0.99)
2	Scraping Loop	Prompt injection forces agent to crawl customer records. Sequence: lookup_customer x 5	Injected
			(P >= 0.85)
3	Path Deviation	Prompt injection forces email of finances without guidelines check. Sequence: search_documents -> calculator -> send_email	Injected
			(P >= 0.98)

Simulation Outcomes & Dashboard Visualization

When an attack is triggered in the sandbox, the Next.js visualizer displays:

- The exact step-by-step tool sequence of the agent.
- The anomaly probability percentage computed by the backend.
- Detailed logs outlining which explainability rules triggered the status override.
- Active alerts populated on the security console to notify operators immediately.

5. Market Comparison & Deployment

Kavalar differs significantly from general LLM guardrails by targeting the behavioral layer of agent workflows rather than the lexical semantic content of user inputs.

Feature	Traditional (e.g. Llama Guard)	Self-Checking (e.g. NeMo)	Kavalar (Our Solution)
Latency Overhead	Moderate (100ms - 300ms)	High (500ms - 2s)	Extremely Low (< 5ms)
Token Cost	Low (input token check)	High (nested LLM calls)	Zero (numerical feature model)
Defeat Method	Adversarial text bypass	Context escape bypass	Tool deviations are flagged
Explainability	Opaque classification label	LLM-generated text	Structured logic override logs
Integration Level	Input/Output wrapper	Agent framework state	Out-of-band middleware sidecar

6. Verification & Quick Start

The submission includes automated deployment scripts to make verification seamless:

1. Standard Local Runner:

- Run './run_local.sh' (Unix/macOS) or 'run_local.bat' (Windows).
- The script creates the virtual environment, installs dependencies, trains the baseline anomaly detector, and runs both FastAPI (port 8000) and Next.js (port 3000) servers concurrently.

2. Docker Compose Runner:

- Run 'docker-compose up --build' from the root directory.
- Automatically spins up the backend and frontend in containers, runs the baseline training sequence, and links the application endpoints seamlessly.